Exercise 2.2 | Linear Regression using sklearn

Do the following tasks:

- Use your fuel dataset
- Perform univariate (single independent variable, choose the highest correlation)
- Make sure to include the R squared, MSE, and RMSE
- Submit the accomplished Jupyter Notebook file in pdf format and share the link that shows the building of linear regression using sklearn.

Name: Abon, Benedict Aldous A. Course / Section: CPE 312 / CPE31S3 Date Performed: 8/6/26 Date Finished: 8/6/26 Istructor: Engr. Robin Valenzuela

Setup

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Use Fuel Dataset (MPG (miles per gallon))

```
# Import df
df = pd.read_csv('auto-mpg.csv')
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 398,\n  \"fields\": [\
n    {\n      \"column\": \"mpg\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 7.815984312565782,\n
\"min\": 9.0,\n        \"max\": 46.6,\n        \"num_unique_values\":
129,\n        \"samples\": [\n          17.7,\n          30.5,\n
30.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"cylinders\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1,\n        \"min\": 3,\n
\"max\": 8,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          4,\n          5,\n          6\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"displacement\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
104.26983817119581,\n        \"min\": 68.0,\n        \"max\": 455.0,\n
\"num_unique_values\": 82,\n        \"samples\": [\n          122.0,\n
307.0,\n          360.0\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"horsepower\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 94,\n
```

```
\"samples\": [\n          \"112\",\n          \"?\",\n
\"78\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"weight\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 846,\n        \"min\": 1613,\n        \"max\": 5140,\n
\"num_unique_values\": 351,\n        \"samples\": [\n          3730,\n
1995,\n          2215\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"acceleration\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2.7576889298126757,\n        \"min\":
8.0,\n        \"max\": 24.8,\n        \"num_unique_values\": 95,\n
\"samples\": [\n          14.7,\n          18.0,\n          14.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"model year\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3,\n        \"min\": 70,\n        \"max\": 82,\n
\"num_unique_values\": 13,\n        \"samples\": [\n          81,\n
79,\n          70\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"origin\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1,\n
3,\n          2\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"car
name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 305,\n        \"samples\": [\n          \"mazda
rx-4\",\n          \"ford f108\",\n          \"buick century luxus
(sw)\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


Describe df

```
df.shape
```

```
(398, 9)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
```

```
 6   model year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car name      398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
```

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"mpg\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 133.37523833494922,\n
\"min\": 7.815984312565782,\n        \"max\": 398.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
23.514572864321607,\n          23.0,\n          398.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"cylinders\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
139.0071020301553,\n        \"min\": 1.7010042445332094,\n
\"max\": 398.0,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          398.0,\n          5.454773869346734,\n
8.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"displacement\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 143.57617465667641,\n        \"min\":
68.0,\n        \"max\": 455.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          193.42587939698493,\n          148.5,\n
398.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"weight\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1535.5522437115258,\n        \"min\": 398.0,\n
\"max\": 5140.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          2970.424623115578,\n          2803.5,\n
398.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"acceleration\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 135.93788360342714,\n        \"min\":
2.7576889298126757,\n        \"max\": 398.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
15.568090452261307,\n          15.5,\n          398.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"model year\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
120.24225119423292,\n        \"min\": 3.697626646732623,\n
\"max\": 398.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          76.01005025125629,\n          76.0,\n
398.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"origin\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 140.19214256834297,\n        \"min\": 0.8020548777266163,\n
\"max\": 398.0,\n        \"num_unique_values\": 6,\n
```

```
\"samples\": [\n          398.0,\n          1.5728643216080402,\n
3.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Data cleaning

```
# Clean df
df_clean = df.copy()

# Ensure column names do not use spaces
df_clean.columns = df_clean.columns.str.replace(' ', '_')

# Remove duplication rows
df_clean.drop_duplicates(inplace=True)

# Replace '?' values with NaN
df_clean = df_clean.replace('?', np.nan)

# Drop empty rows
df_clean.dropna(inplace=True)

# Convert the 'horsepower' column to integer type
df_clean['horsepower'] = df_clean['horsepower'].astype(int)

# car_name to str
df_clean['car_name'] = df_clean['car_name'].astype(str)

df_clean
```


```
{"summary":"{\n  \"name\": \"df_clean\",\n  \"rows\": 392,\n
\"fields\": [\n    {\n      \"column\": \"mpg\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
7.805007486571799,\n        \"min\": 9.0,\n        \"max\": 46.6,\n
\"num_unique_values\": 127,\n        \"samples\": [\n          17.5,\n
35.1,\n          28.8\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"cylinders\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1,\n        \"min\": 3,\n
\"max\": 8,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          4,\n          5,\n          6\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"displacement\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
104.64400390890465,\n        \"min\": 68.0,\n        \"max\": 455.0,\n
\"num_unique_values\": 81,\n        \"samples\": [\n          116.0,\n
307.0,\n          360.0\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"horsepower\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 38,\n        \"min\": 46,\n
\"max\": 230,\n        \"num_unique_values\": 93,\n
\"samples\": [\n          92,\n          100,\n          52\
```

n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"weight\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 849,\n   \"min\": 1613,\n   \"max\": 5140,\n
\"num_unique_values\": 346,\n   \"samples\": [\n   2472,\n
3221,\n   2700\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"acceleration\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 2.7588641191880816,\n   \"min\":
8.0,\n   \"max\": 24.8,\n   \"num_unique_values\": 95,\n
\"samples\": [\n   14.7,\n   18.0,\n   14.3\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"model_year\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
3,\n   \"min\": 70,\n   \"max\": 82,\n
\"num_unique_values\": 13,\n   \"samples\": [\n   81,\n
79,\n   70\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"origin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0,\n   \"min\": 1,\n   \"max\": 3,\n
\"num_unique_values\": 3,\n   \"samples\": [\n   1,\n
3,\n   2\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"car_name\",\n   \"properties\": {\n   \"dtype\":
\"string\",\n   \"num_unique_values\": 301,\n   \"samples\":
[\n   \"subaru dl\",\n   \"buick century limited\",\n
\"plymouth horizon\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df_clean"}

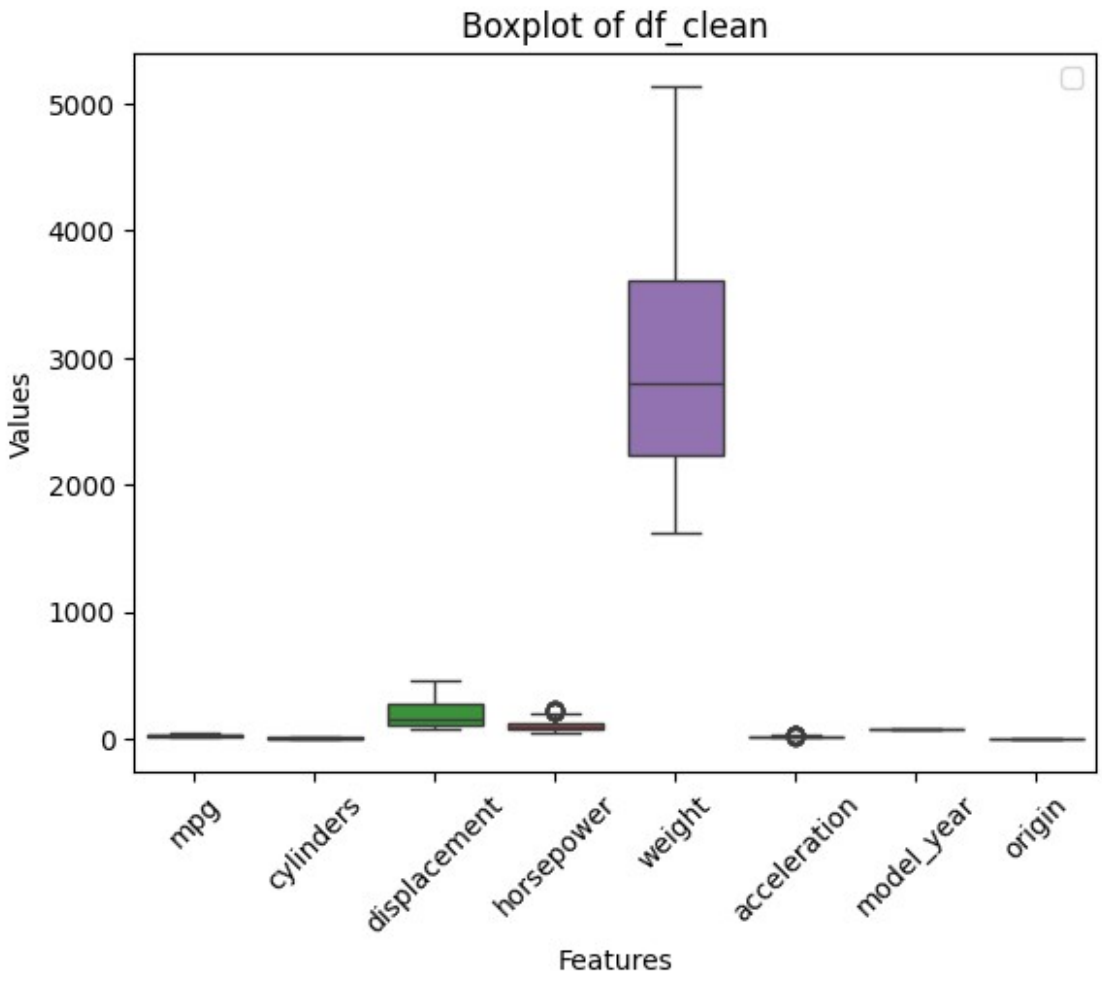
```
# Check for outliers

# Plot
sns.boxplot(df_clean)

# Properties
plt.title('Boxplot of df_clean')
plt.xlabel('Features')
plt.xticks(rotation=45)
plt.ylabel('Values')
plt.legend(loc='upper right')

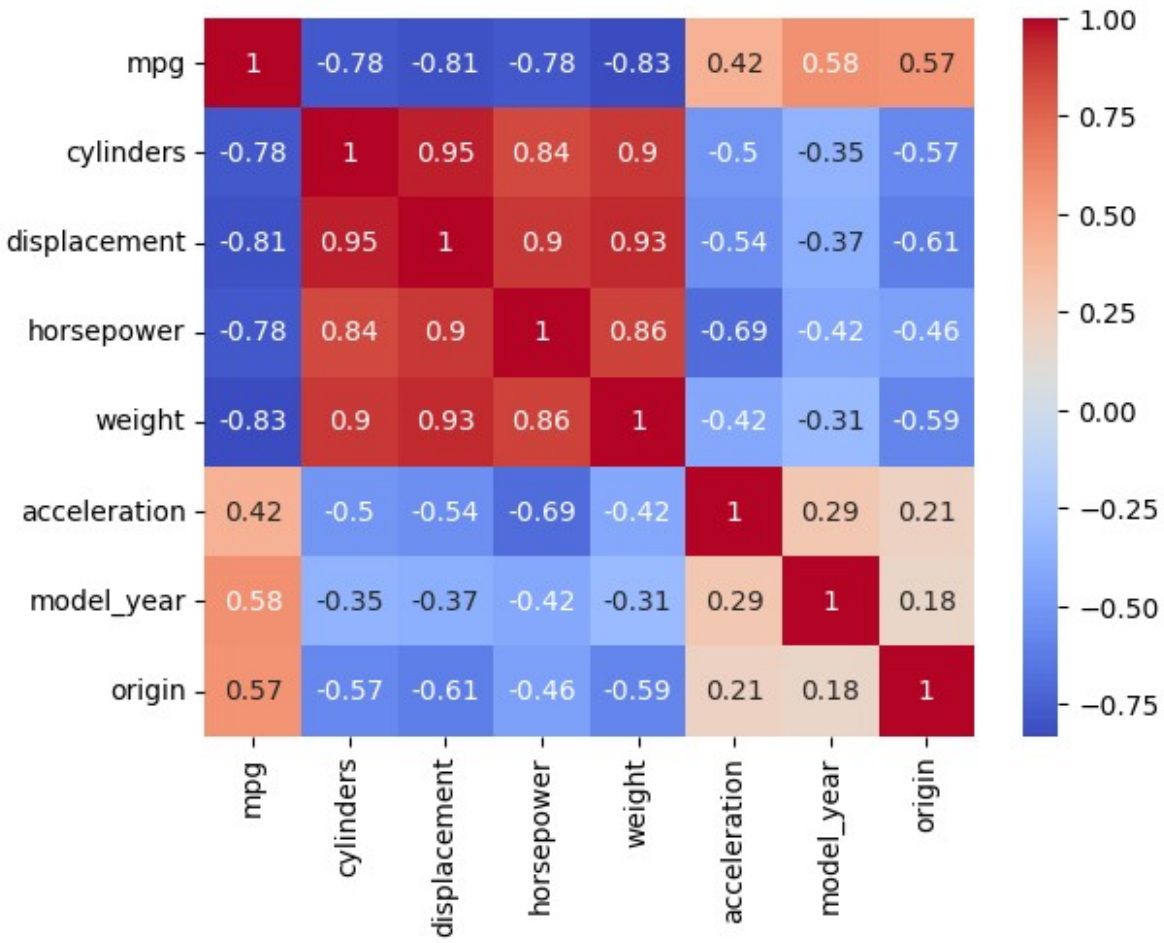
plt.show()
```

```
/tmp/ipykernel_7288/438797667.py:11: UserWarning: No artists with
labels found to put in legend.  Note that artists whose label start
with an underscore are ignored when legend() is called with no
argument.
  plt.legend(loc='upper right')
```



Perform univariate (single independent variable, choose the highest correlation)

```
# Correlation
numeric_df_clean = df_clean.select_dtypes(include='number')
sns.heatmap(numeric_df_clean.corr(), annot=True, cmap='coolwarm')
plt.show()
```



Highest Correlation Columns (0.93) X = weight (independent) y = displacement (dependent)

```
# Import sklearn functions
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# Extract X and y
X = df_clean[['weight']]
y = df_clean['displacement']

# Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Fitting linear regression to the training set
regressor = LinearRegression()
regressor.fit(X_train, y_train)

LinearRegression()
```

```
# Predicting test set results
y_pred = regressor.predict(X_test)
y_pred
```

```
array([104.32487544, 173.65252384,  59.82247407,  77.55479338,
       173.08051354, 100.4352054 , 396.85094304, 113.019432  ,
       227.30709002,  67.4874121 , 294.34669722, 189.09680195,
       352.69174785, 102.26563836, 244.00979079, 151.91613243,
       208.77395628, 122.74360711, 159.35226633, 139.90391612,
       177.99980212,  97.57515389,  80.98685518, 254.76358443,
        78.47000986, 113.019432  , 194.70250289, 247.32745053,
        76.98278308, 141.16233878, 361.84391266, 221.12937877,
       266.3181925 , 155.34819423, 364.5895621 ,  82.13087578,
       384.49552055,  97.11754565, 345.14121189, 381.86427317,
       322.71840811,  95.05830857,  80.41484488, 117.02350411,
       386.66915969, 395.59252038, 229.3663271 ,  97.57515389,
       128.57811217,  75.4955563 , 329.12492348, 114.50665878,
       112.90502994, 101.0072157 , 178.22860624, 269.40704812,
       222.38780143, 187.26636899, 125.60365861, 108.78655578,
       420.30336536, 206.82912126, 162.78432813, 192.18565757,
       141.61994702, 107.8713393 , 165.07236934,  75.4955563 ,
       238.86169808, 350.51810871, 129.37892659, 322.94721223,
       175.94056504,  96.88874153, 250.30190409, 101.46482394,
       320.88797515, 246.64103817,  97.57515389])
```

```
# Coefficient
regressor.coef_
```

```
array([0.11440206])
```

```
# Intercept
regressor.intercept_
```

```
np.float64(-146.1012340577699)
```

Make sure to include the R squared, MSE, and RMSE

```
# Evaluate model using R squared
model_score = r2_score(y_test, y_pred)
print(f"R squared: {model_score:.2f}")
```

```
R squared: 0.91
```

```
# MSE
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.2f}")
```

```
MSE: 1066.08
```

```
# RMSE
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.2f}")
```

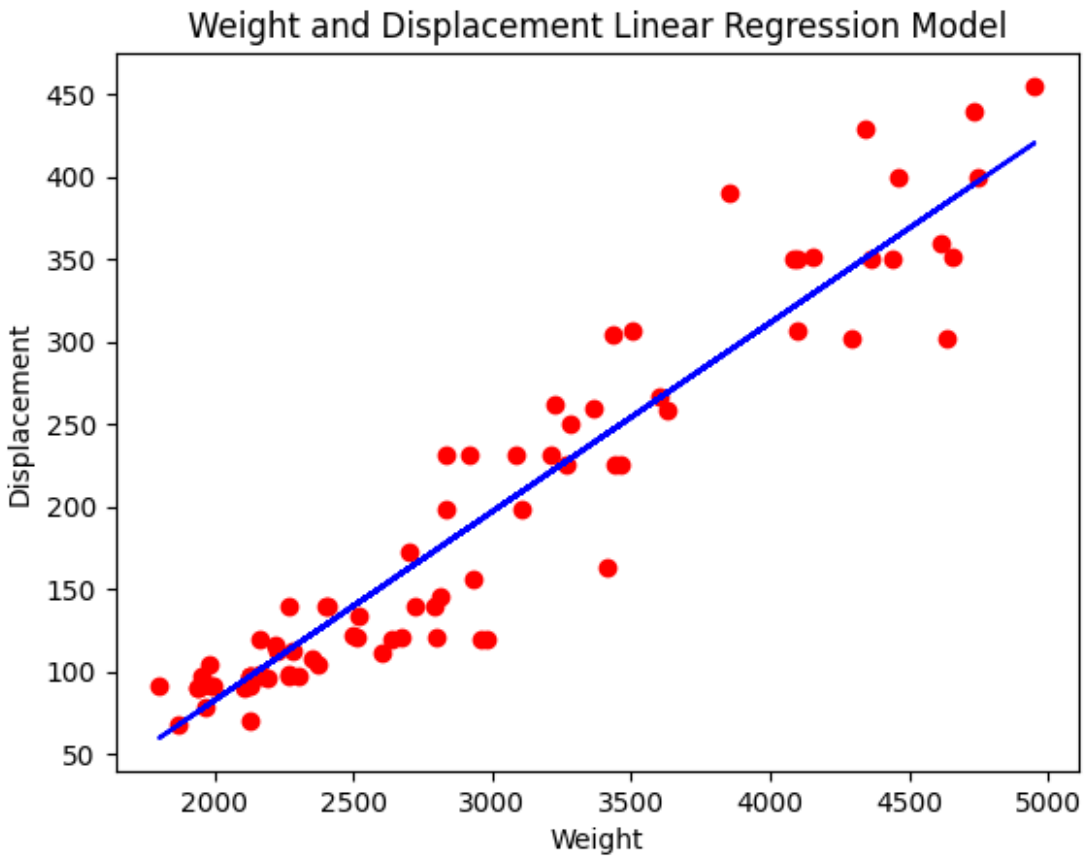
```
RMSE: 32.65
```

Linear Regression Graph

```
# Plot
plt.scatter(X_test, y_test, color='red')
plt.plot(X_test, y_pred, color='blue')

# Properties
plt.title('Weight and Displacement Linear Regression Model')
plt.xlabel('Weight')
plt.ylabel('Displacement')
```

```
Text(0, 0.5, 'Displacement')
```



Fit Error Graph

```
# Residuals
residuals = y_test - y_pred
```

```
# Plot
plt.scatter(y_pred, residuals, color='green')
plt.axhline(y=0, color='red', linestyle='--')

# Properties
plt.title('Fit Error Graph of Linear Regression Model')
plt.xlabel('Predicted Displacement')
plt.ylabel('Residuals')

plt.show()
```

